

Pitch to Postgres: Designing and Querying a Relational Database for FIFA World Cup Data

Milestone 5: Views (05_views.ipynb)

DS604 Introduction to Data Management. Group: Devanshi Dudhatra (202618027), Pujita Sunapu (202618003), Rahul Saha (202618037)

Data: Joshua C. Fjelstul, *The Fjelstul World Cup Database*, <https://github.com/jfjelstul/worldcup>, CC-BY-SA 4.0. Coverage: the 21 men's World Cups from 1930 to 2018.

A **view** is a saved, named `SELECT`. It stores no rows; each time it is queried, PostgreSQL runs its definition against the current tables, so it is always up to date and cannot disagree with the data. `sql/views.sql` defines seven views, built from the strongest Milestone 4 queries:

View	One row per	Built from
<code>v_match_results</code>	match	the joins of Q07/Q12/Q18; rebuilds score text and result labels
<code>v_top_scorers</code>	goal scorer (per person)	Q10
<code>v_tournament_summary</code>	tournament	Q01, Q11, Q17, Q23
<code>v_team_performance</code>	team	Q01, Q02
<code>v_referee_discipline</code>	referee	Q22
<code>v_host_performance</code>	host nation per tournament	Q03
<code>v_head_to_head</code>	(team, opponent) pair	Q07

`sql/views.sql` uses `CREATE OR REPLACE VIEW` only (no `DROP`), so it is safe to run on the course server. This copy was executed against the **local** test database.

```
In [1]: import sys
from pathlib import Path
import pandas as pd

ROOT = Path.cwd() if (Path.cwd() / "src").exists() else Path.cwd().parent
sys.path.insert(0, str(ROOT / "src"))
import db, sql_blocks

pd.set_option("display.width", 130)
pd.set_option("display.max_columns", 40)
pd.set_option("display.max_colwidth", 45)
engine = db.get_engine()
print("Target database:", db.describe())
```

Target database: postgres@localhost:5432/worldcup schema: (default search_path)

1. Create the views from `sql/views.sql`

```
In [2]: with engine.begin() as conn:
        conn.exec_driver_sql((ROOT / "sql" / "views.sql").read_text(encoding="utf-8"))

views = pd.read_sql_query("""
```

```

SELECT table_name AS view_name FROM information_schema.views
WHERE table_schema = current_schema() ORDER BY table_name""", engine)
views["rows"] = [pd.read_sql_query(f"SELECT COUNT(*) AS n FROM {v}", engine).n[0]
for v in views.view_name]
print(views.to_string(index=False))

```

```

view_name  rows
v_head_to_head  1230
v_host_performance  22
v_match_results  900
v_referee_discipline  380
v_team_performance  84
v_top_scorers  1297
v_tournament_summary  21

```

2. v_match_results

Purpose: One readable row per match: tournament, stage, venue, team names, score text, result, winner (including shoot-out winners) and how far the match went. It rebuilds the derived columns (score text, result labels) that the normalised schema does not store.

Based on: matches, with the joins used in Q07, Q12, Q18

SQL concepts: 6-table JOIN (teams joined twice under different aliases), CASE, string concatenation

Expected result: 900 rows (one per match).

```

CREATE OR REPLACE VIEW v_match_results AS
SELECT m.match_id,
       tr.year,
       st.stage_number,
       st.stage_name,
       m.group_name,
       m.match_date,
       m.match_time,
       s.stadium_name,
       s.city_name,
       s.country_name AS
stadium_country,
       h.team_name AS
home_team,
       a.team_name AS
away_team,
       m.home_team_score,
       m.away_team_score,
       CAST(m.home_team_score AS VARCHAR(2)) || '-' ||
CAST(m.away_team_score AS VARCHAR(2))
       || CASE WHEN m.penalty_shootout
              THEN '(' || CAST(m.home_team_score_penalties AS
VARCHAR(2)) || '-'
              || CAST(m.away_team_score_penalties AS
VARCHAR(2)) || ' pens)'
              ELSE '' END AS
score,
       CASE WHEN m.home_team_score > m.away_team_score THEN 'home win'
            WHEN m.home_team_score < m.away_team_score THEN 'away win'

```

```

ELSE 'draw' END
result,
CASE WHEN m.home_team_score > m.away_team_score
OR m.home_team_score_penalties > m.away_team_score_penalties
THEN h.team_name
WHEN m.home_team_score < m.away_team_score
OR m.home_team_score_penalties < m.away_team_score_penalties
THEN a.team_name
END
winner_incl_shootout, -- NULL = drawn, no shoot-out
CASE WHEN m.penalty_shootout THEN 'penalties'
WHEN m.extra_time THEN 'extra time'
ELSE '90 minutes' END
went_to,
m.replayed,
m.replay
FROM matches m
JOIN tournaments tr ON tr.tournament_id = m.tournament_id
JOIN tournament_stages st ON st.tournament_id = m.tournament_id AND
st.stage_number = m.stage_number
JOIN stadiums s ON s.stadium_id = m.stadium_id
JOIN teams h ON h.team_id = m.home_team_id
JOIN teams a ON a.team_id = m.away_team_id;

```

```

In [3]: df = pd.read_sql_query("SELECT * FROM v_match_results ORDER BY match_date DESC,
match_id DESC LIMIT 10", engine)
print(df)

```

	match_id	year	stage_number	stage_name	group_name	match_date	match_time	stadium_name	\
0	M-2018-64	2018	6	final	None	2018-07-15	18:00:00	Luzhniki Stadium	
1	M-2018-63	2018	5	third-place match	None	2018-07-14	17:00:00	Krestovsky Stadium	
2	M-2018-62	2018	4	semi-finals	None	2018-07-11	21:00:00	Luzhniki Stadium	
3	M-2018-61	2018	4	semi-finals	None	2018-07-10	21:00:00	Krestovsky Stadium	
4	M-2018-60	2018	3	quarter-finals	None	2018-07-07	21:00:00	Fisht Olympic Stadium	
5	M-2018-59	2018	3	quarter-finals	None	2018-07-07	18:00:00	Samara Arena	
6	M-2018-58	2018	3	quarter-finals	None	2018-07-06	21:00:00	Kazan Arena	
7	M-2018-57	2018	3	quarter-finals	None	2018-07-06	17:00:00	Nizhny Novgorod Stadium	
8	M-2018-56	2018	2	round of 16	None	2018-07-03	21:00:00	Otkritie Arena	
9	M-2018-55	2018	2	round of 16	None	2018-07-03	17:00:00	Krestovsky Stadium	

	city_name	stadium_country	home_team	away_team	home_team_score	away_team_score	score	result
0	Moscow	Russia	France	Croatia	4	2	4-2	home win
1	Saint Petersburg	Russia	Belgium	England	2	0	2-0	home win
2	Moscow	Russia	Croatia	England	2	1	2-1	home win
3	Saint Petersburg	Russia	France	Belgium	1	0	1-0	home win
4	Sochi	Russia	Russia	Croatia	2	2	2-2 (3-4 pens)	draw
5	Samara	Russia	Sweden	England	0	2	0-2	away win
6	Kazan	Russia	Brazil	Belgium	1	2	1-2	away win
7	Nizhny Novgorod	Russia	Uruguay	France	0	2	0-2	away win
8	Moscow	Russia	Colombia	England	1	1	1-1 (3-4 pens)	draw
9	Saint Petersburg	Russia	Sweden	Switzerland	1	0	1-0	home win

	winner_incl_shootout	went_to	replayed	replay
0	France	90 minutes	False	False
1	Belgium	90 minutes	False	False
2	Croatia	extra time	False	False
3	France	90 minutes	False	False
4	Croatia	penalties	False	False
5	England	90 minutes	False	False
6	Belgium	90 minutes	False	False
7	France	90 minutes	False	False
8	England	penalties	False	False
9	Sweden	90 minutes	False	False

The latest matches first: the 2018 final, third-place match and semi-finals, with readable names, score text and the winner.

Example use: All matches decided by penalties in finals and semi-finals.

```
In [4]: q = """
SELECT year, stage_name, home_team, score, away_team, winner_incl_shootout
FROM v_match_results
WHERE went_to = 'penalties' AND stage_name IN ('final', 'semi-finals')
ORDER BY year;
"""

print(pd.read_sql_query(q, engine).to_string(index=False))
```

year	stage_name	home_team	score	away_team	winner_incl_shootout
1982	semi-finals	West Germany	3-3 (5-4 pens)	France	West Germany
1990	semi-finals	Argentina	1-1 (4-3 pens)	Italy	Argentina
1990	semi-finals	West Germany	1-1 (4-3 pens)	England	West Germany
1994	final	Brazil	0-0 (3-2 pens)	Italy	Brazil
1998	semi-finals	Brazil	1-1 (4-2 pens)	Netherlands	Brazil
2006	final	Italy	1-1 (5-3 pens)	France	Italy
2014	semi-finals	Netherlands	0-0 (2-4 pens)	Argentina	Argentina

3. v_top_scorers

Purpose: One row per player who scored at least one goal (own goals excluded), with an all-time rank. Counted per PERSON, so a player who played for two teams (e.g. West Germany and Germany) gets one combined row.

Based on: Q10 (all-time top scorers)

SQL concepts: JOIN, WHERE, GROUP BY, window function RANK(), STRING_AGG(DISTINCT ...)

Expected result: 1,297 rows (one per scorer); goal_rank 1 = Miroslav Klose (16).

```
CREATE OR REPLACE VIEW v_top_scorers AS
SELECT RANK() OVER (ORDER BY COUNT(*) DESC) AS goal_rank,
       p.player_id,
       COALESCE(p.given_name || ' ', '') || p.family_name AS player,
       STRING_AGG(DISTINCT t.team_name, ' / ') AS teams,
       COUNT(*) AS goals,
       SUM(CASE WHEN g.penalty THEN 1 ELSE 0 END) AS
penalty_goals,
       COUNT(DISTINCT g.tournament_id) AS
tournaments_scored_in,
       COUNT(DISTINCT g.match_id) AS
matches_scored_in,
       MIN(tr.year) AS
first_goal_year,
       MAX(tr.year) AS
last_goal_year
FROM goals g
JOIN players p ON p.player_id = g.player_id
JOIN teams t ON t.team_id = g.player_team_id
JOIN tournaments tr ON tr.tournament_id = g.tournament_id
WHERE NOT g.own_goal
GROUP BY p.player_id, p.given_name, p.family_name;
```

```
In [5]: df = pd.read_sql_query("SELECT * FROM v_top_scorers ORDER BY goal_rank, player LIMIT
10", engine)
print(df)
```

goal_rank	player_id	player	teams	goals	penalty_goals	tournaments_scored_in	\
0	1	P-05239	Miroslav Klose	Germany	16	0	4
1	2	P-04601	Ronaldo	Brazil	15	1	3
2	3	P-02483	Gerd Müller	West Germany	14	1	2
3	4	P-00643	Just Fontaine	France	13	0	1
4	5	P-03795	Pelé	Brazil	12	0	4
5	6	P-00876	Jürgen Klinsmann	Germany / West Germany	11	0	3
6	6	P-07498	Sándor Kocsis	Hungary	11	0	1
7	8	P-05392	Gabriel Batistuta	Argentina	10	4	3
8	8	P-02777	Gary Lineker	England	10	2	2
9	8	P-07200	Grzegorz Lato	Poland	10	0	3

matches_scored_in	first_goal_year	last_goal_year
0	11	2002
1	11	1998
2	9	1970
3	6	1958
4	8	1958
5	10	1990
6	4	1954
7	6	1994
8	6	1986
9	8	1974

The all-time ranking. Jürgen Klinsmann's row combines his West Germany and Germany goals, because the view counts per person.

Example use: Scorers who netted at four different World Cups.

```
In [6]: q = """
SELECT player, teams, goals, tournaments_scored_in, first_goal_year, last_goal_year
FROM v_top_scorers
WHERE tournaments_scored_in >= 4
ORDER BY goals DESC, player;
"""
print(pd.read_sql_query(q, engine).to_string(index=False))
```

player	teams	goals	tournaments_scored_in	first_goal_year	last_goal_year
Miroslav Klose	Germany	16	4	2002	2014
Pelé	Brazil	12	4	1958	1970
Uwe Seeler	West Germany	9	4	1958	1970
Cristiano Ronaldo	Portugal	7	4	2006	2018

4. v_tournament_summary

Purpose: A fact sheet for each tournament: hosts, format, number of teams and matches, the final four, goals, top scorer(s), cards and shoot-outs. It rebuilds the host, winner and format information that the normalised tournaments table does not store.

Based on: Q01 (standings), Q11 (top scorer), Q17 (goals per match), Q23 (cards)

SQL concepts: several CTEs, window function RANK(), LEFT JOINs (no cards before 1970 gives NULL, not 0), correlated scalar subqueries, STRING_AGG with ORDER BY, EXISTS

Expected result: 21 rows (1930-2018).

```
CREATE OR REPLACE VIEW v_tournament_summary AS
WITH match_stats AS (
    SELECT tournament_id,
           COUNT(*) AS
matches,
           SUM(home_team_score + away_team_score) AS
goals,
           SUM(CASE WHEN extra_time THEN 1 ELSE 0 END) AS
```

```

extra_time_matches,
    SUM(CASE WHEN penalty_shootout THEN 1 ELSE 0 END) AS
shootouts
    FROM matches
    GROUP BY tournament_id
),
card_stats AS (
    SELECT tournament_id,
        COUNT(*)
AS cards,
    SUM(CASE WHEN red_card OR second_yellow_card THEN 1 ELSE 0 END)
AS sendings_off
    FROM bookings
    GROUP BY tournament_id
),
scorer_tally AS (
    SELECT tournament_id, player_id, COUNT(*) AS goals,
        RANK() OVER (PARTITION BY tournament_id ORDER BY COUNT(*) DESC)
AS rnk
    FROM goals
    WHERE NOT own_goal
    GROUP BY tournament_id, player_id
),
top_scorers AS (
    SELECT st.tournament_id,
        STRING_AGG(COALESCE(p.given_name || ' ', '') || p.family_name, ', '
    ORDER BY p.family_name) AS top_scorers,
        MAX(st.goals) AS top_scorer_goals
    FROM scorer_tally st
    JOIN players p ON p.player_id = st.player_id
    WHERE st.rnk = 1
    GROUP BY st.tournament_id
)
SELECT tr.year,
    tr.tournament_name,
    tr.start_date,
    tr.end_date,
    (SELECT STRING_AGG(t.team_name, ' & ' ORDER BY t.team_name)
    FROM host_countries h JOIN teams t ON t.team_id = h.team_id
    WHERE h.tournament_id = tr.tournament_id)
AS hosts,
    (SELECT STRING_AGG(s.stage_name, ' > ' ORDER BY s.stage_number)
    FROM tournament_stages s WHERE s.tournament_id = tr.tournament_id)
AS format,
    (SELECT COUNT(*) FROM qualified_teams q WHERE q.tournament_id =
tr.tournament_id) AS teams,
    ms.matches,
    (SELECT t.team_name FROM tournament_standings s JOIN teams t ON
t.team_id = s.team_id
    WHERE s.tournament_id = tr.tournament_id AND s.position = 1)
AS champion,
    (SELECT t.team_name FROM tournament_standings s JOIN teams t ON
t.team_id = s.team_id
    WHERE s.tournament_id = tr.tournament_id AND s.position = 2)
AS runner_up,
    (SELECT t.team_name FROM tournament_standings s JOIN teams t ON

```

```

t.team_id = s.team_id
    WHERE s.tournament_id = tr.tournament_id AND s.position = 3)
AS third_place,
    (SELECT t.team_name FROM tournament_standings s JOIN teams t ON
t.team_id = s.team_id
    WHERE s.tournament_id = tr.tournament_id AND s.position = 4)
AS fourth_place,
    CASE WHEN EXISTS (SELECT 1 FROM host_countries h
        JOIN tournament_standings s ON s.tournament_id =
h.tournament_id AND s.team_id = h.team_id
        WHERE h.tournament_id = tr.tournament_id AND
s.position = 1)
    THEN TRUE ELSE FALSE END
AS host_won,
    ms.goals,
    ROUND(1.0 * ms.goals / ms.matches, 2)
AS goals_per_match,
    ts.top_scorers,
    ts.top_scorer_goals,
    ms.extra_time_matches,
    ms.shootouts,
    cs.cards,
-- NULL before 1970 (no cards)
    cs.sendings_off,
    ROUND(1.0 * cs.cards / ms.matches, 2)
AS cards_per_match
FROM tournaments tr
JOIN match_stats ms ON ms.tournament_id = tr.tournament_id
LEFT JOIN card_stats cs ON cs.tournament_id = tr.tournament_id
LEFT JOIN top_scorers ts ON ts.tournament_id = tr.tournament_id;

```

```

In [7]: df = pd.read_sql_query("SELECT * FROM v_tournament_summary ORDER BY year DESC LIMIT
10", engine)
print(df)

```

matches	champion	runner_up	third_place	fourth_place	host_won	goals	goals_per_match	\
0	64	France	Croatia	Belgium	England	False	169	2.64
1	64	Germany	Argentina	Netherlands	Brazil	False	171	2.67
2	64	Spain	Netherlands	Germany	Uruguay	False	145	2.27
3	64	Italy	France	Germany	Portugal	False	147	2.30
4	64	Brazil	Germany	Turkey	South Korea	False	161	2.52
5	64	France	Brazil	Croatia	Netherlands	True	171	2.67
6	52	Brazil	Italy	Sweden	Bulgaria	False	141	2.71
7	52	West Germany	Argentina	Italy	England	False	115	2.21
8	52	Argentina	West Germany	France	Belgium	False	132	2.54
9	52	Italy	West Germany	Poland	France	False	146	2.81

\	top_scorers	top_scorer_goals	extra_time_matches	shootouts	cards	sendings_off
0	Harry Kane	6	5	4	225	4
1	James Rodríguez	6	8	4	191	10
2	Diego Forlán, Thomas Müller, Wesley Sneijder	5	4	2	270	17
3	Miroslav Klose	5	6	4	353	27
4	Ronaldo	8	5	2	282	16
5	Davor Šuker	6	4	3	276	22
6	Oleg Salenko, Hristo Stoichkov	6	4	3	243	15
7	Salvatore Schillaci	6	8	4	184	16
8	Gary Lineker	6	5	3	145	8
9	Paolo Rossi	6	1	1	104	5

cards_per_match
0
3.52
1
2.98
2
4.22
3
5.52
4
4.41
5
4.31
6
4.67
7
3.54
8
2.79
9
2.00

One fact sheet per World Cup, newest first. The format column rebuilds the stage sequence that the normalised tournaments table no longer stores as flags.

Example use: Tournaments won by the host nation.

```
In [8]: q = """
SELECT year, hosts, champion, runner_up, goals_per_match
FROM v_tournament_summary
WHERE host_won
ORDER BY year;
"""

print(pd.read_sql_query(q, engine).to_string(index=False))
```

year	hosts	champion	runner_up	goals_per_match
1930	Uruguay	Uruguay	Argentina	3.89
1934	Italy	Italy	Czechoslovakia	4.12
1966	England	England	West Germany	2.78
1974	West Germany	West Germany	Netherlands	2.55
1978	Argentina	Argentina	Netherlands	2.68
1998	France	France	Brazil	2.67

5. v_team_performance

Purpose: One row per national team: participation span, match record (shoot-out = draw), goals, win percentage, titles, finals, top-four finishes and best finish.

Based on: Q01 (titles, finals, top four) and Q02 (all-time records)

SQL concepts: three aggregating CTEs combined with JOIN and LEFT JOIN, COALESCE for teams that never reached the top four, CASE to label the best finish

Expected result: 84 rows (one per team); Brazil: 109 matches, 73 won, 5 titles.

```
CREATE OR REPLACE VIEW v_team_performance AS
WITH record AS (
    SELECT team_id,
           COUNT(*)
    AS matches,
           SUM(CASE WHEN goals_for > goals_against THEN 1 ELSE 0 END)
    AS won,
           SUM(CASE WHEN goals_for = goals_against THEN 1 ELSE 0 END)
    AS drawn,
           SUM(CASE WHEN goals_for < goals_against THEN 1 ELSE 0 END)
    AS lost,
           SUM(goals_for)
    AS goals_for,
           SUM(goals_against)
    AS goals_against
    FROM team_appearances
    GROUP BY team_id
),
participation AS (
    SELECT q.team_id,
           COUNT(*) AS tournaments,
           MIN(tr.year) AS first_year,
           MAX(tr.year) AS last_year
    FROM qualified_teams q
    JOIN tournaments tr ON tr.tournament_id = q.tournament_id
    GROUP BY q.team_id
),
finishes AS (
    SELECT team_id,
           SUM(CASE WHEN position = 1 THEN 1 ELSE 0 END) AS titles,
           SUM(CASE WHEN position <= 2 THEN 1 ELSE 0 END) AS finals,
           COUNT(*) AS top_four,
           MIN(position) AS best_position
    FROM tournament_standings
    GROUP BY team_id
)
SELECT t.team_id,
```

```

t.team_name,
t.team_code,
c.confederation_code,
p.tournaments,
p.first_year,
p.last_year,
r.matches, r.won, r.drawn, r.lost,
r.goals_for, r.goals_against,
r.goals_for - r.goals_against           AS goal_diff,
ROUND(100.0 * r.won / r.matches, 1)     AS win_pct,
COALESCE(f.titles, 0)                   AS titles,
COALESCE(f.finals, 0)                   AS finals,
COALESCE(f.top_four, 0)                 AS top_four,
CASE f.best_position WHEN 1 THEN 'champion' WHEN 2 THEN 'runner-up'
                      WHEN 3 THEN 'third' WHEN 4 THEN 'fourth'
                      ELSE 'outside top four' END AS best_finish

FROM teams t
JOIN confederations c ON c.confederation_id = t.confederation_id
JOIN participation p ON p.team_id = t.team_id
JOIN record r ON r.team_id = t.team_id
LEFT JOIN finishes f ON f.team_id = t.team_id;

```

```

In [9]: df = pd.read_sql_query("SELECT * FROM v_team_performance ORDER BY titles DESC,
win_pct DESC LIMIT 10", engine)
print(df)

```

	team_id	team_name	team_code	confederation_code	tournaments	first_year	last_year	matches	won	drawn	lost	goals_for	goals_against
0	T-09	Brazil	BRA	CONMEBOL	21	1930	2018	109	73	18	18	105	124
1	T-39	Italy	ITA	UEFA	18	1934	2014	83	45	21	17	77	51
2	T-82	West Germany	DEU	UEFA	10	1954	1990	62	36	14	12	77	54
3	T-03	Argentina	ARG	CONMEBOL	17	1930	2018	81	43	15	23	93	44
4	T-28	France	FRA	UEFA	15	1930	2018	66	34	13	19	77	43
5	T-80	Uruguay	URY	CONMEBOL	13	1930	2018	56	24	12	20	74	13
6	T-29	Germany	DEU	UEFA	9	1934	2018	47	31	6	10	48	47
7	T-70	Spain	ESP	UEFA	15	1934	2018	63	30	15	18	72	27
8	T-27	England	ENG	UEFA	15	1950	2018	69	29	21	19	64	27
9	T-46	Netherlands	NLD	UEFA	10	1934	2014	50	27	12	11	48	38

	goals_against	goal_diff	win_pct	titles	finals	top_four	best_finish
0	105	124	67.0	5	7	11	champion
1	77	51	54.2	4	6	8	champion
2	77	54	58.1	3	6	8	champion
3	93	44	53.1	2	5	5	champion
4	77	43	51.5	2	3	6	champion
5	74	13	42.9	2	2	5	champion
6	48	47	66.0	1	2	5	champion
7	72	27	47.6	1	1	2	champion
8	64	27	42.0	1	1	3	champion
9	48	38	54.0	0	3	5	runner-up

Teams ordered by titles, then win percentage: the eight champions come first.

Example use: Best record among teams that never reached the top four (at least 10 matches).

In [10]:

```
q = """
SELECT team_name, tournaments, matches, won, drawn, lost, win_pct
FROM v_team_performance
WHERE top_four = 0 AND matches >= 10
ORDER BY win_pct DESC
LIMIT 5;
"""
print(pd.read_sql_query(q, engine).to_string(index=False))
```

team_name	tournaments	matches	won	drawn	lost	win_pct
Denmark	5	20	9	5	6	45.0
Colombia	6	22	9	3	10	40.9
Ecuador	3	10	4	1	5	40.0
Romania	7	21	8	5	8	38.1
Ghana	3	12	4	3	5	33.3

6. v_referee_discipline

Purpose: One row per referee: matches refereed, tournaments, and (for matches since 1970, when cards were introduced) cards, sendings-off and cards per match.

Based on: Q22 (strictest referees)

SQL concepts: CTE with LEFT JOIN (card-free matches count 0), GROUP BY, conditional aggregation, NULLIF to avoid division by zero (referees who worked only before 1970 get NULL)

Expected result: 380 rows (one per referee); with at least 5 matches since 1970, the highest cards_per_match is Lubos Michel (8.00), as in Q22.

```
CREATE OR REPLACE VIEW v_referee_discipline AS
WITH per_match AS (
    SELECT ra.referee_id, ra.match_id, tr.year,
           COUNT(b.booking_id)
    AS cards,
           SUM(CASE WHEN b.red_card OR b.second_yellow_card THEN 1 ELSE 0
    END) AS sendings_off
    FROM referee_appearances ra
    JOIN tournaments tr ON tr.tournament_id = ra.tournament_id
    LEFT JOIN bookings b ON b.match_id = ra.match_id
    GROUP BY ra.referee_id, ra.match_id, tr.year
)
SELECT r.referee_id,
       COALESCE(r.given_name || ' ', '') || r.family_name AS
referee,
       r.country_name AS
country,
       c.confederation_code,
       COUNT(*) AS
matches,
       COUNT(DISTINCT pm.year) AS
tournaments,
       MIN(pm.year) AS
first_year,
       MAX(pm.year) AS
last_year,
       SUM(CASE WHEN pm.year >= 1970 THEN 1 ELSE 0 END) AS
```

```

matches_since_1970,
    CAST(SUM(pm.cards) AS INTEGER) AS
cards,
    CAST(COALESCE(SUM(pm.sendings_off), 0) AS INTEGER) AS
sendings_off,
    ROUND(1.0 * SUM(pm.cards) / NULLIF(SUM(CASE WHEN pm.year >= 1970 THEN
1 ELSE 0 END), 0), 2)

AS

cards_per_match
FROM per_match pm
JOIN referees r ON r.referee_id = pm.referee_id
JOIN confederations c ON c.confederation_id = r.confederation_id
GROUP BY r.referee_id, r.given_name, r.family_name, r.country_name,
c.confederation_code;

```

```

In [11]: df = pd.read_sql_query("SELECT * FROM v_referee_discipline ORDER BY cards_per_match
DESC NULLS LAST, matches DESC LIMIT 10", engine)
print(df)

```

	referee_id	referee	country	confederation_code	matches	tournaments	first_year	last_year	\
0	R-197	Antonio López Nieto	Spain	UEFA	1	1	2002	2002	
1	R-367	Jan Wegereef	Netherlands	UEFA	1	1	2002	2002	
2	R-337	John Toro Rendón	Colombia	CONMEBOL	1	1	1998	1998	
3	R-155	Valentin Ivanov	Russia	UEFA	3	1	2006	2006	
4	R-096	Jesús Díaz	Colombia	CONMEBOL	2	1	1986	1986	
5	R-236	Ľuboš Micheľ	Slovakia	UEFA	5	2	2002	2006	
6	R-008	Khalil Al-Ghamdi	Saudi Arabia	AFC	2	1	2010	2010	
7	R-299	Ángel Sánchez	Argentina	CONMEBOL	2	1	2002	2002	
8	R-001	Essam Abdel-Fatah	Egypt	CAF	1	1	2006	2006	
9	R-257	Pierluigi Pairetto	Italy	UEFA	1	1	1994	1994	

	matches_since_1970	cards	sendings_off	cards_per_match
0	1	16	2	16.00
1	1	12	0	12.00
2	1	10	3	10.00
3	3	26	4	8.67
4	2	17	3	8.50
5	5	40	2	8.00
6	2	16	1	8.00
7	2	14	2	7.00
8	1	7	0	7.00
9	1	7	0	7.00

Without a minimum number of matches, the top of the list is referees with one or two very heated matches. That is why Q22 used HAVING matches >= 5, which a user can apply to the view with a WHERE clause.

Example use: Referees with the most World Cup matches.

```

In [12]: q = """
SELECT referee, country, confederation_code, matches, tournaments, first_year,
last_year, cards_per_match
FROM v_referee_discipline
ORDER BY matches DESC, referee
LIMIT 5;
"""
print(pd.read_sql_query(q, engine).to_string(index=False))

```

	referee	country	confederation_code	matches	tournaments	first_year	last_year	cards_per_match
	Ravshan Irmatov	Uzbekistan	AFC	11	3	2010	2018	4.00
	Néstor Pitana	Argentina	CONMEBOL	9	2	2014	2018	2.56
	Benito Archundia	Mexico	CONCACAF	8	2	2006	2010	3.50
	Joël Quiniou	France	UEFA	8	3	1986	1994	4.88
	Jorge Larrión	Uruguay	CONMEBOL	8	2	2006	2010	3.88

7. v_host_performance

Purpose: One row per host nation per tournament: matches, wins, draws, losses, goals, stage reached, final position and whether it won the title. Measures host advantage tournament by tournament.

Based on: Q03 (host-country advantage)

SQL concepts: JOIN through the composite key (tournament_id, team_id), LEFT JOIN to standings (most hosts did not finish in the top four), aggregating subquery in FROM, CASE

Expected result: 22 rows (21 tournaments; 2002 had two hosts); 6 hosts won the title.

```
CREATE OR REPLACE VIEW v_host_performance AS
SELECT tr.year,
       t.team_name                                AS host,
       rec.matches, rec.won, rec.drawn, rec.lost,
       rec.goals_for, rec.goals_against,
       q.performance                             AS
stage_reached,
       s.position                                AS
final_position, -- NULL outside top four
       CASE s.position WHEN 1 THEN 'champion' WHEN 2 THEN 'runner-up'
                WHEN 3 THEN 'third' WHEN 4 THEN 'fourth'
                ELSE 'outside top four' END        AS finish,
       CASE WHEN s.position = 1 THEN TRUE ELSE FALSE END AS
won_title
FROM host_countries h
JOIN tournaments tr      ON tr.tournament_id = h.tournament_id
JOIN teams t            ON t.team_id = h.team_id
JOIN qualified_teams q    ON q.tournament_id = h.tournament_id AND q.team_id
= h.team_id
JOIN (SELECT tournament_id, team_id,
              COUNT(*)                                AS
matches,
              SUM(CASE WHEN goals_for > goals_against THEN 1 ELSE 0 END) AS
won,
              SUM(CASE WHEN goals_for = goals_against THEN 1 ELSE 0 END) AS
drawn,
              SUM(CASE WHEN goals_for < goals_against THEN 1 ELSE 0 END) AS
lost,
              SUM(goals_for)                                AS
goals_for,
              SUM(goals_against)                            AS
goals_against
      FROM team_appearances
      GROUP BY tournament_id, team_id) rec
      ON rec.tournament_id = h.tournament_id AND
rec.team_id = h.team_id
LEFT JOIN tournament_standings s
      ON s.tournament_id = h.tournament_id AND s.team_id
= h.team_id;
```

```
In [13]: df = pd.read_sql_query("SELECT * FROM v_host_performance ORDER BY year DESC LIMIT 10", engine)
print(df)
```

	year	host	matches	won	drawn	lost	goals_for	goals_against	stage_reached	final_position \
0	2018	Russia	5	2	2	1	11	7	quarter-finals	NaN
1	2014	Brazil	7	3	2	2	11	14	third-place match	4.0
2	2010	South Africa	3	1	1	1	3	5	group stage	NaN
3	2006	Germany	7	5	1	1	14	6	third-place match	3.0
4	2002	South Korea	7	3	2	2	8	6	third-place match	4.0
5	2002	Japan	4	2	1	1	5	3	round of 16	NaN
6	1998	France	7	6	1	0	15	2	final	1.0
7	1994	United States	4	1	1	2	3	4	round of 16	NaN
8	1990	Italy	7	6	1	0	10	2	third-place match	3.0
9	1986	Mexico	5	3	2	0	6	2	quarter-finals	NaN

	finish	won_title
0	outside top four	False
1	fourth	False
2	outside top four	False
3	third	False
4	fourth	False
5	outside top four	False
6	champion	True
7	outside top four	False
8	third	False
9	outside top four	False

Each host's record in its own tournament, newest first; final_position is NULL when the host did not finish in the top four.

Example use: Hosts that did not reach the top four.

```
In [14]: q = """
SELECT year, host, matches, won, drawn, lost, stage_reached
FROM v_host_performance
WHERE final_position IS NULL
ORDER BY year;
"""
print(pd.read_sql_query(q, engine).to_string(index=False))
```

year	host	matches	won	drawn	lost	stage_reached
1938	France	2	1	0	1	quarter-finals
1954	Switzerland	4	2	0	2	quarter-finals
1970	Mexico	4	2	1	1	quarter-finals
1982	Spain	5	1	2	2	second group stage
1986	Mexico	5	3	2	0	quarter-finals
1994	United States	4	1	1	2	round of 16
2002	Japan	4	2	1	1	round of 16
2010	South Africa	3	1	1	1	group stage
2018	Russia	5	2	2	1	quarter-finals

8. v_head_to_head

Purpose: One row per (team, opponent) pair that has met at a World Cup, from the first team's point of view: matches, wins, draws, losses, goals and first/last meeting. Filter by two team names to get any head-to-head instantly (each pair appears twice, once from each side).

Based on: Q07 (head-to-head, self-join)

SQL concepts: JOIN of team_appearances to teams twice (team and opponent), GROUP BY on the pair, conditional aggregation

Expected result: one row per ordered pair (1,230 rows = 615 pairs x 2); Brazil v Sweden: 7 matches, 5 wins, 2 draws.

```

CREATE OR REPLACE VIEW v_head_to_head AS
SELECT ta.team_id,
       t.team_name
team,
       ta.opponent_id,
       o.team_name
opponent,
       COUNT(*)
matches,
       SUM(CASE WHEN ta.goals_for > ta.goals_against THEN 1 ELSE 0 END) AS
wins,
       SUM(CASE WHEN ta.goals_for = ta.goals_against THEN 1 ELSE 0 END) AS
draws,
       SUM(CASE WHEN ta.goals_for < ta.goals_against THEN 1 ELSE 0 END) AS
losses,
       SUM(ta.goals_for)
goals_for,
       SUM(ta.goals_against)
goals_against,
       MIN(tr.year)
first_meeting,
       MAX(tr.year)
last_meeting
FROM team_appearances ta
JOIN teams t      ON t.team_id = ta.team_id
JOIN teams o      ON o.team_id = ta.opponent_id
JOIN tournaments tr ON tr.tournament_id = ta.tournament_id
GROUP BY ta.team_id, t.team_name, ta.opponent_id, o.team_name;

```

```

In [15]: df = pd.read_sql_query("SELECT * FROM v_head_to_head ORDER BY matches DESC, team,
opponent LIMIT 10", engine)
print(df)

```

	team_id	team	opponent_id	opponent	matches	wins	draws	losses	goals_for	goals_against	first_meeting
\											
0	T-09	Brazil	T-71	Sweden	7	5	2	0	21	8	1938
1	T-71	Sweden	T-09	Brazil	7	0	2	5	8	21	1938
2	T-03	Argentina	T-27	England	5	1	1	3	5	8	1962
3	T-03	Argentina	T-39	Italy	5	0	3	2	4	6	1974
4	T-03	Argentina	T-46	Netherlands	5	1	2	2	4	7	1974
5	T-03	Argentina	T-48	Nigeria	5	5	0	0	9	4	1994
6	T-09	Brazil	T-20	Czechoslovakia	5	3	2	0	10	4	1938
7	T-09	Brazil	T-39	Italy	5	2	1	2	9	7	1938
8	T-09	Brazil	T-44	Mexico	5	4	1	0	13	0	1950
9	T-09	Brazil	T-46	Netherlands	5	1	1	3	5	10	1974
last_meeting											
0	1994										
1	1994										
2	2002										
3	1990										
4	2014										
5	2018										
6	1970										
7	1994										
8	2018										
9	2014										

The most frequent fixtures. Each pair appears twice, once from each team's point of view.

Example use: Any pair in one line: England against West Germany and Germany.

```

In [16]: q = ""
SELECT team, opponent, matches, wins, draws, losses, goals_for, goals_against,

```

```

first_meeting, last_meeting
FROM v_head_to_head
WHERE team = 'England' AND opponent IN ('West Germany', 'Germany')
ORDER BY opponent;
"""
print(pd.read_sql_query(q, engine).to_string(index=False))

```

team	opponent	matches	wins	draws	losses	goals_for	goals_against	first_meeting	last_meeting
England	Germany	1	0	0	1	1	4	2010	2010
England	West Germany	4	1	2	1	7	6	1966	1990

9. Do the views agree with the Milestone 4 queries?

Each view is compared with the query it was built from (run fresh from `sql/queries.sql`). They must give identical figures.

```

In [17]: Q = {b["id"]: pd.read_sql_query(b["sql"], engine) for b in sql_blocks.parse(ROOT /
"sql" / "queries.sql")}
    if b["id"] in ("Q01", "Q02", "Q03", "Q07", "Q10", "Q17", "Q22")}
rv = lambda s: pd.read_sql_query(s, engine)
tests = []

tp = rv("SELECT team_name, matches AS played, won, drawn, lost, goals_for,
goals_against, goal_diff, win_pct "
"FROM v_team_performance WHERE matches >= 10")
q2 = Q["Q02"]
tests.append(("v_team_performance = Q02 (all-time records, 49 teams)",

tp.sort_values("team_name").reset_index(drop=True).equals(q2[tp.columns].sort_values
("team_name").reset_index(drop=True))))
t1 = rv("SELECT team_name, titles, finals, top_four FROM v_team_performance WHERE
top_four > 0")
tests.append(("v_team_performance = Q01 (titles, finals, top four)",
t1.sort_values("team_name").reset_index(drop=True).equals(Q["Q01"]
[t1.columns].sort_values("team_name").reset_index(drop=True))))
ts = rv("SELECT goal_rank, player, goals FROM v_top_scorers WHERE goal_rank <= 10
ORDER BY goal_rank, player")
q10 = Q["Q10"][["rank", "player", "goals"]].rename(columns={"rank": "goal_rank"})
tests.append(("v_top_scorers rank <= 10 = Q10 (13 players, ties kept)",
ts.reset_index(drop=True).equals(q10.reset_index(drop=True))))
s = rv("SELECT year, goals, goals_per_match FROM v_tournament_summary ORDER BY
year")
tests.append(("v_tournament_summary goals = Q17",
s.reset_index(drop=True).equals(Q["Q17"][["year", "goals", "goals_per_match"]]))
h = rv("SELECT * FROM v_head_to_head WHERE team = 'Brazil' AND opponent =
'Sweden').iloc[0]
last = Q["Q07"].iloc[-1]
tests.append(("v_head_to_head Brazil-Sweden = Q07 running totals",
(h.wins, h.draws, h.losses) == (last.a_wins_so_far, last.draws_so_far,
last.b_wins_so_far)))
rd = rv("SELECT referee, matches_since_1970 AS matches, cards, sendings_off,
cards_per_match "
"FROM v_referee_discipline WHERE matches_since_1970 >= 5 ORDER BY
cards_per_match DESC, matches DESC")
tests.append(("v_referee_discipline (>= 5 matches) = Q22",
rd.sort_values("referee").reset_index(drop=True).equals(Q["Q22"]
[rd.columns].sort_values("referee").reset_index(drop=True))))
hp = rv("SELECT SUM(matches) AS m, SUM(won) AS w FROM v_host_performance").iloc[0]
q3 = Q["Q03"].query("side == 'host' and era == 'all years'").iloc[0]

```

```

tests.append(("v_host_performance totals = Q03 (118 matches, 62.7% won)",
             int(hp.m) == q3.matches and round(100 * hp.w / hp.m, 1) ==
             q3.win_pct))
mr = rv("SELECT COUNT(*) AS n, SUM(home_team_score + away_team_score) AS goals, "
        "SUM(CASE WHEN went_to = 'penalties' THEN 1 ELSE 0 END) AS shootouts FROM
v_match_results").iloc[0]
tests.append(("v_match_results: 900 matches, 2548 goals, 30 shoot-outs", (mr.n,
mr.goals, mr.shootouts) == (900, 2548, 30)))

res = pd.DataFrame(tests, columns=["check", "agrees"])
print(res.to_string(index=False))
assert res.agrees.all()
print(f"\n{res.agrees.sum()} / {len(res)} views agree with their source queries.")

```

	check	agrees
v_team_performance = Q02 (all-time records, 49 teams)	True	True
v_team_performance = Q01 (titles, finals, top four)	True	True
v_top_scorers rank <= 10 = Q10 (13 players, ties kept)	True	True
v_tournament_summary goals = Q17	True	True
v_head_to_head Brazil-Sweden = Q07 running totals	True	True
v_referee_discipline (>= 5 matches) = Q22	True	True
v_host_performance totals = Q03 (118 matches, 62.7% won)	True	True
v_match_results: 900 matches, 2548 goals, 30 shoot-outs	True	True

8 / 8 views agree with their source queries.

10. Summary: why views, and when to use a table instead

- **Reuse without repetition.** Analysts, broadcasters and the final report can run `SELECT ... FROM v_team_performance WHERE ...` instead of copying a 40-line query, and every user gets the same definitions (for example, a shoot-out counts as a draw).
- **Normalised storage, readable output.** The tables store each fact once (Milestone 2). Views such as `v_match_results` and `v_tournament_summary` put back the readable columns that were deliberately not stored: score text, result labels, hosts, champions and format.
- **Always correct.** A view is recomputed on every query, so it can never fall out of date; a summary *table* would have to be rebuilt after every change to the data.
- **When a table would be better.** If a summary were expensive and queried very often on a large database, a `MATERIALIZED VIEW` (stored results, refreshed on demand) or a summary table would be faster. With 57,269 rows every view here returns in milliseconds, so plain views are the right choice.